

TechWise 360

Comprehensive Pre-Defense Technical Examination Guide & Reference

Target Hardware: Meta Quest 2 (OpenXR) • Engine: Unity 6000.4.8f1 (URP 17.4.0, XRI 3.4.1) • Backend: Supabase • Local Lab System

1. Executive Project Summary & Defense Pitch

TechWise 360 is an immersive Virtual Reality (VR) computer hardware assembly and disassembly training simulation system specifically engineered for the standalone Meta Quest 2 headset. In technical vocational education, providing individual, hands-on PC hardware labs is restricted by tight equipment budgets, mechanical wear, and component destruction through electrostatic discharge (ESD) or improper force.

The 60-Second Opening Elevator Pitch (Memorize This)

"Good morning, esteemed panelists. We present **TechWise 360**, a high-fidelity standalone VR simulation system designed to bridge the gap between computer hardware theory and practical physical assembly. Running natively on the Meta Quest 2 through OpenXR and Unity 6, TechWise 360 delivers an authentic 1:1 scale PC building experience. Unlike basic drag-and-drop simulations, our system incorporates true mechanical locking latches, thermal paste application, screw threading physics with full-view magnification, orientation-aware guidance, and comprehensive assessment tracking backed by an offline-resilient Cloud Web Portal."

2. How Virtual Reality Operates in TechWise 360

Panelists frequently interrogate candidates on the scientific and technological fundamentals of VR. Be prepared to explain these exact principles:

A. 6 Degrees of Freedom (6DoF) & Real-Time Tracking

- **Rotational Tracking (Orientation):** **Pitch** (looking up/down), **Yaw** (turning left/right), and **Roll** (tilting head side-to-side). Handled at over 1000 Hz by the headset's internal **Inertial Measurement Unit (IMU)** containing gyroscopes and accelerometers.
- **Positional Tracking (Translation):** **X** (strafing left/right), **Y** (ducking/standing), and **Z** (walking forward/backward).
- **Inside-Out Optical Tracking (SLAM):** The Meta Quest 2 uses 4 wide-angle infrared monochrome cameras placed at the corners of the visor. Using **Simultaneous Localization and Mapping (SLAM)**, the tracking system identifies static high-contrast features in the physical room (corners, doorframes, desk lines) and calculates the headset's real-world translation relative to those points.

- **Code Architecture:** In Unity, our VR Player Rig uses an `XROrigin`. Attached to the `Main Camera` is a `TrackedPoseDriver`. Every frame, the OpenXR subsystem polls the physical headset's fused IMU/optical pose and applies exact 3D coordinates to the virtual camera's Transform.

B. Stereoscopic 3D Depth Perception

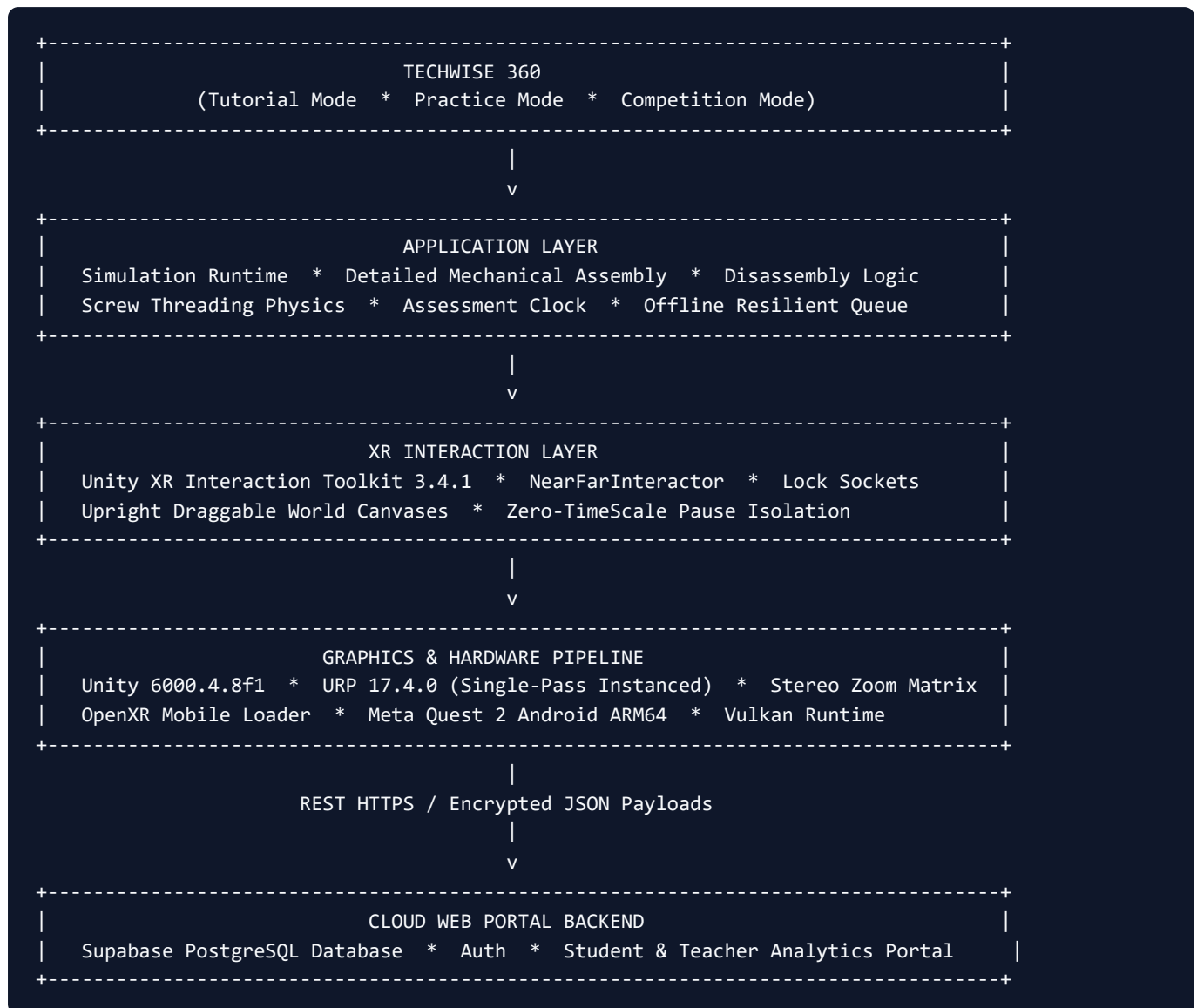
Humans perceive depth through **stereoscopic vision** and **retinal disparity**. Because our eyes are spaced approximately 63 mm apart (Interpupillary Distance or IPD), each eye views an object from a slightly different perspective angle.

In TechWise 360, Unity renders the scene twice per frame—generating a separate projection matrix and viewport for the Left Eye and the Right Eye. When you inspect a motherboard slot or bring a CPU within inches of your face, your brain fuses these two distinct visual streams into one perceived image with true tangible depth, allowing instinctive spatial alignment.

C. Low Latency & High Refresh Rate (90 Hz)

To prevent simulation sickness, **Motion-to-Photon Latency** must remain under 20 milliseconds. TechWise 360 runs in **Vulkan** on Android ARM64 utilizing **Single-Pass Instanced Rendering** in the Universal Render Pipeline (URP). This renders both stereo eye views in a single GPU draw pass, maintaining a stable 90 frames per second without dropouts.

3. System Architecture & Component Pipeline



4. Detailed Breakdown of Core Code Modules

Module 1: Simulation Core & State Machine (`TechWiseSimulationRuntime.cs`)

- **Single Source of Truth:** Manages the active state of all `XRGrabInteractable` computer parts and `XRLockSocketInteractor` sockets on the workbench.
- **Assembly Sequence Enforcement:** Ensures realistic prerequisite order (e.g. CPU cannot be mounted if the socket bracket is closed; CPU cooler cannot be clamped until thermal paste is deposited).
- **Mistake Auditing:** Implements `RecordMistake()`. When a student attempts an invalid connection or drops a component, an error record containing component name, timestamp, and severity is logged without crashing the active simulation.

Module 2: Detailed Mechanical Assembly ([TechWiseDetailedAssemblyRuntime.cs](#))

Unlike simplistic educational games where objects just snap instantly, TechWise 360 simulates fine mechanical procedures:

- **CPU Socket Hinges** ([TechWiseAssemblyHinge.cs](#)): Simulates the two-stage Intel/AMD retention mechanism. Students must swing the load plate ([Cover](#)) and tension arm ([Lever](#)) independently before inserting the processor.
- **M.2 SSD Angular Insertion & Standoff**: Recreates the realistic 30-degree insertion angle into the M.2 keyed slot, followed by pivoting the board flat against the motherboard standoff and securing it with an ultra-fine screw.
- **Thermal Compound Syringe**: Tracks cap removal, syringe plunger depression, and volumetric paste deposition across the Integrated Heat Spreader (IHS) of the CPU.

Module 3: Fastener Physics & URP Stereo Zoom ([TechWiseScrewDetail.cs](#))

- **The Problem Solved**: Viewing micro-fasteners (M.2, cooler brackets) in VR is difficult due to display resolution. Traditional Picture-in-Picture (PiP) inset windows ruin immersion and cause severe rendering lag by doubling draw calls.
- **URP Rendering Hook**: Directly hooks into the Universal Render Pipeline:
[RenderPipelineManager.beginCameraRendering += OnBeginCameraRendering;](#)
- **Stereo Matrix Scaling**: While driving screws, it dynamically scales the projection matrix diagonal elements:

```
left.m00 *= currentZoom; left.m11 *= currentZoom;  
right.m00 *= currentZoom; right.m11 *= currentZoom;  
cam.SetStereoProjectionMatrix(Camera.StereoscopicEye.Left, left);  
cam.SetStereoProjectionMatrix(Camera.StereoscopicEye.Right, right);
```

- **User Experience**: The student enjoys smooth 1.6x full-field magnification while driving screws, **without** shifting camera position, **without** altering physical geometry, and **preserving 100% natural 6DoF head tracking**.

Module 4: Orientation-Aware Directional Arrows (`TechWiseComponentGeometry.cs`)

- **Dynamic Normal Evaluation:** Evaluates socket orientation relative to world and chassis vectors:
 - **Flat Upward Sockets (RAM, CPU, M.2):** Evaluates `vertical > 0.5f` to project arrows descending straight downward into the motherboard.
 - **Side-Facing Sockets (GPU in PCIe, Motherboard into Case, Side SSD):** Calculates the horizontal chassis plane normal, rendering guide arrows approaching horizontally from the open chassis side.
- **Chassis Rotation Adaptation:** Because vectors are evaluated relative to parent coordinate space, when the player spins the PC case around on the desk, the guidance arrows seamlessly rotate with the case.

Module 5: In-Game Pause Session & Draggable UI (`TechWiseInGameMenu.cs`)

- **Interaction Layer Isolation:** When paused (`Time.timeScale = 0`), standard VR games suffer from a bug where clicking UI accidentally grabs objects behind the canvas. Our custom `TechWisePauseSession` zeroes controller `hand.interactionLayers = 0` and suspends locomotion providers, ensuring crisp button clicks with zero background interference.
- **Upright Draggable Canvases (`TechWiseDraggableUiPanel.cs`):** World-space UI canvases can be repositioned by grabbing their header bar. Pitch and roll rotations are locked to `0.0f`, keeping panels strictly upright and legible. When released, user repositioning flags prevent the system from snapping panels back during step advancements.

Module 6: Assessment Engine & Resilient Offline Queue (`TechWiseOfflineAttemptQueue.cs`)

- **Grading Equation:**
$$\text{Final Grade} = \max\left(0, 100 - (\text{Mistakes} \times \text{Weight}) - \text{Time Deductions}\right)$$
- **Offline-First Architecture:** School Wi-Fi networks are notoriously unreliable. If a student completes an evaluation while offline, the payload is serialized into persistent local storage as encrypted JSON. When connectivity returns, a background synchronization daemon automatically posts records to the Supabase portal database.

5. Comparison of Simulation Modes

Feature	Tutorial Mode	Practice Mode	Competition Mode
Target Audience	Novices & First-Time VR Users	Students Building Proficiency	Formal Practical Evaluation
Visual Guidance	High (Bounding boxes, 3D outlines, dynamic arrows)	Moderate (Component status feedback)	None (Zero hints, arrows, or outlines)
Controller Hints	Callout rings on physical buttons	None	None
Timer & Scoring	Untimed / Ungraded	Live timer, mistake counter displayed	Hidden timer & mistakes, scored on portal
Reset Practice	Skip lesson option	Instant Reset in Pause Menu	Disabled (Strict attempt integrity)

6. Anticipated Panel Questions & High-Scoring Defense Answers

Q1: Why did you choose standalone Meta Quest 2 and OpenXR over PCVR (SteamVR / Oculus Link)?

Answer: "We selected the standalone Meta Quest 2 because it dramatically lowers institutional deployment barriers. High-end PCVR setups require expensive dedicated gaming workstations (\$1,500+ per student) and create dangerous cable trip hazards in computer classrooms. By developing natively for Android ARM64 through the **OpenXR standard**, our application is lightweight, untethered, and future-proof. OpenXR ensures that our interaction and rendering architecture can be deployed to Meta Quest 3, HTC Vive, or Pico headsets with zero rewrites to core simulation code."

Q2: How do your virtual controllers interact with UI in 3D world space?

Answer: "We employ Unity XRI's `NearFarInteractor` coupled with `TrackedDeviceGraphicRaycaster` on our world-space canvases. When the controller ray crosses the canvas plane, `TrackedDeviceEventData` calculates the exact 2D hit coordinate on the `RectTransform` and triggers standard Unity Event System interfaces like `IPointerClickHandler` and `IPointerEnterHandler`. To prevent false negatives, we programmatically set `checkFor3DOcclusion = false` on modal canvases so workbench geometry never blocks menu interaction."

Q3: How do you handle physics stability so computer components do not jitter or fly away?

Answer: "Physical jitter in VR usually stems from physics conflicts between the player's direct interactor and socket triggers. We solved this with custom `XRLockSocketInteractor` components that use kinematic transitions. Once a component enters tolerance bounds and is released, we instantly set its `Rigidbody` to `isKinematic = true`, cancel residual linear/angular momentum, and smoothly parent it to the socket attach transform. If an item is dropped out of bounds or falls off the workbench, our `TechWiseComponentRecovery` service detects the collision floor and teleports it back to its original tray."

Q4: What ensures academic integrity if a student cheats by taking off the headset?

Answer: "The Meta Quest 2 contains a hardware optical proximity sensor between the lenses. The moment the headset is removed, the Android OS fires an `onPause()` signal, halting the OpenXR session. In our assessment engine, `TechWiseAssessmentClock` utilizes unscaled monotonic system time and compares it against active focus intervals. An attempt cannot be exploited by removing the headset to pause the evaluation timer, and attempt integrity flags are flagged in the upload payload."

Q5: How does this simulation prove educational effectiveness?

Answer: "Our instructional design follows cognitive scaffolding theory. In **Tutorial Mode**, cognitive load is minimized through active visual cues and controller callouts. In **Practice Mode**, students develop muscle memory and procedural familiarity with real-time feedback. Finally, in **Competition Mode**, students demonstrate independent mastery under realistic conditions. Objective metrics (time to assemble, mistake frequency, fastener errors) are aggregated in the Web Portal, providing instructors with concrete diagnostic data on student learning deficiencies."

7. Defense Day Demonstration Script & Checklist

1. Tutorial Demo (2 mins):

- Show how controller rings guide user fingers to Trigger, Grip, and Joysticks.
- Grab the floating tutorial header and drag it to a comfortable side angle, showing it remains strictly upright and stays in place.

2. Practice Mode Mechanics (3 mins):

- Open the CPU socket latch (cover & lever hinges), place the CPU, and lock the latch.
- Deposit thermal paste on the CPU IHS.
- Drive cooler screws with the screwdriver, highlighting the smooth 1.6x URP full-view magnification.
- Press the **Left Menu Button** to open the Pause Menu: click **Settings**, toggle the wrist shortcut, and show that workbench items behind the menu cannot be accidentally grabbed.

3. Competition Mode & Web Portal (2 mins):

- Start Competition mode: point out that hints and arrows are strictly disabled.
- Complete or submit the session, then immediately open the Web Portal on a browser laptop to show the completed score, time, and mistake log updated in real time.